

LAPORAN PRAKTIKUM DAN TUGAS

STRUKTUR DATA

“LINKEDLIST”

DISUSUN OLEH:

Aliyatar Rafi Ahmad

2511533031

DOSEN PENGAMPU:

Dr. Wahyudi, S.T, M.T

ASISTEN PRAKTIKUM:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum Struktur Data pekan ke-5 yang berjudul *Single Linked List (SLL)* dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum sekaligus sebagai sarana untuk memahami konsep struktur data khususnya Linked List dalam bahasa pemrograman Java.

Pada praktikum pekan ke-5 ini, penulis mempelajari mengenai konsep dasar Linked List, khususnya Single Linked List (SLL), serta implementasinya dalam bentuk program. Melalui praktikum ini, penulis dapat memahami bagaimana proses penyimpanan data secara dinamis menggunakan node yang saling terhubung, serta operasi-operasi dasar yang dapat dilakukan seperti penambahan, penghapusan, dan penelusuran data.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat bagi penulis maupun pembaca dalam memahami materi struktur data, khususnya Linked List.

Padang, 05 Mei 2026

Aliyatar Rafi Ahmad

DAFTAR ISI

KATA PENGANTAR.....	i
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
2.1 Pengertian LinkedList	2
2.2 Operasi Dasar LinkedList	2
2.3 Implementasi dalam Java.....	2
2.3.1 NodeSLL.....	3
2.3.2 PencarianSLL	3
2.3.3 TambahSLL.....	4
2.3.4 HapusSLL	6
BAB III PENUTUP	8
3.1 Kesimpulan	8
DAFTAR PUSTAKA.....	8
LAPORAN TUGAS	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam pemrograman yang digunakan untuk mengatur, menyimpan, dan mengelola data agar dapat diproses secara efisien. Pemilihan struktur data yang tepat sangat berpengaruh terhadap performa program, terutama ketika data yang dikelola bersifat dinamis dan sering mengalami perubahan. Salah satu struktur data yang dapat digunakan untuk mengatasi permasalahan tersebut adalah Linked List.

Single Linked List (SLL) merupakan bentuk sederhana dari Linked List yang terdiri atas sekumpulan node, di mana setiap node menyimpan data dan referensi menuju node berikutnya. Berbeda dengan array yang memiliki ukuran tetap dan penyimpanan memori berurutan, Single Linked List memungkinkan penambahan dan penghapusan data dilakukan dengan lebih fleksibel tanpa perlu menggeser seluruh elemen. Oleh karena itu, pada praktikum pekan ke-5 ini dilakukan implementasi Single Linked List menggunakan bahasa pemrograman Java untuk memahami konsep node serta operasi dasar yang terdapat pada struktur data tersebut.

1.2 Tujuan Praktikum

1. Memahami konsep dasar struktur data Single Linked List.
2. Mempelajari cara kerja node dan hubungan antar node dalam Linked List.
3. Memahami operasi dasar pada Linked List seperti penambahan, penghapusan, dan penelusuran data.

1.3 Manfaat Praktikum

1. Menambah pemahaman tentang struktur data Linked List.
2. Melatih kemampuan dalam mengimplementasikan konsep struktur data ke dalam program.

BAB II

PEMBAHASAN

2.1 Pengertian LinkedList

Single Linked List merupakan salah satu struktur data linear yang terdiri dari sekumpulan node yang saling terhubung. Setiap node pada struktur ini memiliki dua bagian utama, yaitu data yang menyimpan nilai, serta pointer (link) yang menunjuk ke node berikutnya. Berbeda dengan array yang menggunakan penyimpanan memori secara berurutan, Linked List bersifat dinamis karena elemen-elemen disimpan secara tidak berurutan di memori dan dihubungkan melalui referensi. Hal ini memungkinkan penambahan dan penghapusan data dilakukan dengan lebih fleksibel tanpa perlu menggeser elemen lainnya.

2.2 Operasi Dasar LinkedList

Beberapa operasi dasar yang dapat dilakukan pada Single Linked List antara lain:

- **Insertion (Penambahan):** menambahkan node baru ke dalam list, baik di awal, tengah, maupun akhir.
- **Deletion (Penghapusan):** menghapus node tertentu dari list.
- **Traversal (Penelusuran):** menampilkan seluruh elemen dengan cara menelusuri node dari head hingga akhir.

2.3 Implementasi dalam Java

Pada praktikum pekan ke-5 telah dibuat beberapa kode program yang berkaitan dengan implementasi struktur data **LinkedList** dalam bahasa pemrograman Java.

2.3.1 NodeSLL

```

1 package pekan5_2511533031;
2
3 public class NodeSLL_2511533031 {
4     // node bagian data
5     int data_3031;
6     //Pointer ke node berikutnya
7     NodeSLL_2511533031 next_3031;
8     // Konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511533031(int data_3031) {
10         this.data_3031 = data_3031;
11         this.next_3031 = null;
12     }
13 }
14 }

```

Gambar 2.1

Class NodeSLL digunakan untuk merepresentasikan satu node dalam struktur data Single Linked List. Di dalamnya terdapat dua atribut utama, yaitu data untuk menyimpan nilai, dan next sebagai pointer yang menunjuk ke node berikutnya sehingga antar node dapat saling terhubung.

Class ini juga memiliki konstruktor yang berfungsi untuk menginisialisasi nilai data saat node dibuat serta mengatur pointer next bernilai null. Node ini menjadi elemen dasar dalam pembentukan Linked List.

2.3.2 PencarianSLL

```

1 package pekan5_2511533031;
2
3 public class PencarianSLL_2511533031 {
4     static boolean searchKey(NodeSLL_2511533031 head_3031, int key_3031) {
5         NodeSLL_2511533031 curr_3031 = head_3031;
6         while (curr_3031 != null) {
7             if (curr_3031.data_3031 == key_3031)
8                 return true;
9             curr_3031 = curr_3031.next_3031; }
10        return false; }
11    public static void traversal(NodeSLL_2511533031 head_3031) {
12        // mulai dari head
13        NodeSLL_2511533031 curr_3031 = head_3031;
14        // telusuri sampai pointer null
15        while (curr_3031 != null) {
16            System.out.print(" " + curr_3031.data_3031);
17            curr_3031 = curr_3031.next_3031; }
18        System.out.println();
19    public static void main(String[] args) {
20        NodeSLL_2511533031 head_3031 = new NodeSLL_2511533031(14);
21        head_3031.next_3031 = new NodeSLL_2511533031(21);
22        head_3031.next_3031.next_3031 = new NodeSLL_2511533031(13);
23        head_3031.next_3031.next_3031.next_3031 = new NodeSLL_2511533031(30);
24        head_3031.next_3031.next_3031.next_3031.next_3031 = new NodeSLL_2511533031(10);
25        System.out.print("Penelusuran SLL : ");
26        traversal(head_3031);
27        // data yang akan dicari
28        int key_3031 = 30;
29        System.out.print("cari data " + key_3031 + " = ");
30        if (searchKey(head_3031, key_3031))
31            System.out.println("ketemu");
32        else
33            System.out.println("tidak ada");
34    }
35 }

```

Gambar 2.2

Class PencarianSLL digunakan untuk melakukan operasi pencarian dan penelusuran pada struktur data Single Linked List. Method `searchKey()` berfungsi untuk mencari suatu nilai dalam Linked List dengan cara menelusuri node satu per satu mulai dari head hingga akhir. Jika data yang dicari ditemukan, maka method akan mengembalikan nilai `true`, sedangkan jika tidak ditemukan akan mengembalikan `false`.

Selain itu, terdapat method `traversal()` yang digunakan untuk menampilkan seluruh elemen dalam Linked List secara berurutan. Pada method `main`, dibuat beberapa node yang saling terhubung untuk membentuk Linked List, kemudian dilakukan proses penelusuran dan pencarian data. Dengan demikian, program ini menunjukkan bagaimana cara mengakses dan mencari data dalam struktur Single Linked List.

2.3.3 TambahSLL

```

1 package pekan5_2511533031;
2 public class TambahSLL_2511533031 {
3     public static NodeSLL_2511533031 insertAtFront(NodeSLL_2511533031 head_3031, int value_3031) {
4         NodeSLL_2511533031 new_node_3031 = new NodeSLL_2511533031(value_3031);
5         new_node_3031.next_3031 = head_3031;
6         return new_node_3031;
7     }
8     // fungsi menambahkan node di akhir SLL
9     public static NodeSLL_2511533031 insertAtEnd(NodeSLL_2511533031 head_3031, int value_3031) {
10        // buat sebuah node dengan sebuah nilai
11        NodeSLL_2511533031 newNode_3031 = new NodeSLL_2511533031(value_3031);
12        // jika list kosong maka node jadi head
13        if (head_3031 == null) {
14            return newNode_3031;
15        }
16        // simpan head ke variabel sementara
17        NodeSLL_2511533031 last_3031 = head_3031;
18        // telusuri ke node akhir
19        while (last_3031.next_3031 != null) {
20            last_3031 = last_3031.next_3031;
21        }
22        // ubah pointer
23        last_3031.next_3031 = newNode_3031;
24        return head_3031;
25    }
26    static NodeSLL_2511533031 GetNode_3031(int data_3031) {
27        return new NodeSLL_2511533031(data_3031);
28    }
29    static NodeSLL_2511533031 insertPos(NodeSLL_2511533031 headNode_3031, int position_3031, int value_3031) {
30        NodeSLL_2511533031 head_3031 = headNode_3031;
31        if (position_3031 < 1)
32            System.out.print("Invalid position");
33        if (position_3031 == 1) {
34            NodeSLL_2511533031 new_node_3031 = new NodeSLL_2511533031(value_3031);
35            new_node_3031.next_3031 = head_3031;
36            return new_node_3031;
37        } else {
38            while (position_3031-- != 0) {
39                if (position_3031 == 1) {
40                    NodeSLL_2511533031 newNode_3031 = GetNode_3031(value_3031);
41                    newNode_3031.next_3031 = headNode_3031.next_3031;
42                    headNode_3031.next_3031 = newNode_3031;

```

```

43         break;
44     }
45     headNode_3031 = headNode_3031.next_3031;
46     }
47     if (position_3031 != -1)
48         System.out.print("Posisi di luar jangkauan"); }
49     return head_3031;
50 }
51 public static void printList(NodeSLL_2511533031 head_3031) {
52     NodeSLL_2511533031 curr_3031 = head_3031;
53     while (curr_3031.next_3031 != null) {
54         System.out.print(curr_3031.data_3031 + "-->");
55         curr_3031 = curr_3031.next_3031;
56     }
57     if (curr_3031.next_3031 == null) {
58         System.out.print(curr_3031.data_3031);
59     }
60     System.out.println();
61 }
62 public static void main(String[] args) {
63     // buat linked list 2->3->5->6
64     NodeSLL_2511533031 head_3031 = new NodeSLL_2511533031(2);
65     head_3031.next_3031 = new NodeSLL_2511533031(3);
66     head_3031.next_3031.next_3031 = new NodeSLL_2511533031(5);
67     head_3031.next_3031.next_3031.next_3031 = new NodeSLL_2511533031(6);
68
69     // cetak list asli
70     System.out.print("Senarai berantai awal:");
71     printList(head_3031);
72
73     // tambahkan node baru di depan
74     System.out.print("tambah 1 simpul di depan: ");
75     int data_3031 = 1;
76     head_3031 = insertAtFront(head_3031, data_3031);
77
78     // cetak update list
79     printList(head_3031);
80
81     // tambahkan node baru di belakang
82     System.out.print("tambah 1 simpul di belakang: ");
83     int data2_3031 = 7;
84     head_3031 = insertAtEnd(head_3031, data2_3031);
85
86     // cetak update list
87     printList(head_3031);
88     System.out.print("tambah 1 simpul ke data 4: ");
89     int data3_3031 = 4;
90     int pos_3031 = 4;
91     head_3031 = insertPos(head_3031, pos_3031, data3_3031);
92
93     // cetak update list
94     printList(head_3031);
95 }
96 }

```

Gambar 2.3

Class TambahSLL digunakan untuk melakukan operasi penambahan node pada struktur data Single Linked List. Terdapat beberapa method utama, yaitu `insertAtFront()` untuk menambahkan node di bagian awal, `insertAtEnd()` untuk menambahkan node di bagian akhir dengan cara menelusuri hingga node terakhir, serta `insertPos()` untuk menyisipkan node pada posisi tertentu. Setiap method bekerja dengan membuat node baru dan mengatur pointer agar node tersebut terhubung dengan benar dalam Linked List.

Selain itu, terdapat method `printList()` yang berfungsi untuk menampilkan seluruh isi Linked List secara berurutan. Pada method `main`, dibuat Linked List awal kemudian dilakukan beberapa operasi penambahan di depan, belakang, dan posisi tertentu. Program ini menunjukkan bahwa penambahan data pada Linked List dapat dilakukan dengan fleksibel tanpa perlu menggeser elemen seperti pada array.

2.3.4 HapusSLL

```

1 package pekan5_2511533031;
2 public class HapusSLL_2511533031 {
3     // fungsi untuk menghapus head
4     public static NodeSLL_2511533031 deleteHead(NodeSLL_2511533031 head_3031) {
5         // jika SLL kosong
6         if (head_3031 == null)
7             return null;
8         // pindahkan head ke node berikutnya
9         head_3031 = head_3031.next_3031;
10        // Return head baru
11        return head_3031;
12    }
13    // fungsi menghapus node terakhir SLL
14    public static NodeSLL_2511533031 removeLastNode (NodeSLL_2511533031 head_3031) {
15        //jika list kosong, return null
16        if (head_3031 == null) {
17            return null;
18        }
19        //jika list satu node, hapus node dan return null
20        if (head_3031.next_3031 == null) {
21            return null;
22        }
23        //temukan node terakhir ke dua
24        NodeSLL_2511533031 secondLast = head_3031;
25        while (secondLast.next_3031.next_3031 != null) {
26            secondLast = secondLast.next_3031;
27        }
28        // hapus node terakhir
29        secondLast.next_3031 = null;
30        return head_3031;
31    }
32    // fungsi menghapus node di posisi tertentu
33    public static NodeSLL_2511533031 deleteNode(NodeSLL_2511533031 head_3031, int position_3031) {
34        NodeSLL_2511533031 temp_3031 = head_3031;
35        NodeSLL_2511533031 prev_3031 = null;
36        // jika linked list null
37        if (temp_3031 == null)
38            return head_3031;
39        // kasus 1: head dihapus
40        if (position_3031 == 1) {
41            head_3031 = temp_3031.next_3031;
42            return head_3031;
43        }
44        // kasus 2: menghapus node di tengah
45        // telusuri ke node yang dihapus
46        for (int i = 1; temp_3031 != null && i < position_3031; i++) {
47            prev_3031 = temp_3031;
48            temp_3031 = temp_3031.next_3031;
49        }
50        // jika ditemukan, hapus node
51        if (temp_3031 != null) {
52            prev_3031.next_3031 = temp_3031.next_3031;
53        } else {
54            System.out.println("Data tidak ada");
55        }
56        return head_3031;
57    }
58    // fungsi mencetak SLL
59    public static void printList (NodeSLL_2511533031 head_3031) {
60        NodeSLL_2511533031 curr_3031 = head_3031;
61        while (curr_3031.next_3031 != null) {
62            System.out.print(curr_3031.data_3031+"->");
63            curr_3031 = curr_3031.next_3031;
64        }
65        if (curr_3031.next_3031 == null) {
66            System.out.print(curr_3031.data_3031);
67            System.out.println();
68        }
69    }
70    //kelas main
71    public static void main(String[] args) {
72        // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
73        NodeSLL_2511533031 head_3031 = new NodeSLL_2511533031(1);
74        head_3031.next_3031 = new NodeSLL_2511533031(2);
75        head_3031.next_3031.next_3031 = new NodeSLL_2511533031(3);
76        head_3031.next_3031.next_3031.next_3031 = new NodeSLL_2511533031(4);
77        head_3031.next_3031.next_3031.next_3031.next_3031 = new NodeSLL_2511533031(5);
78        head_3031.next_3031.next_3031.next_3031.next_3031.next_3031 = new NodeSLL_2511533031(6);
79        //cetak list awal
80        System.out.println("list awal: ");
81        printList(head_3031);
82        //hapus head
83        head_3031 = deleteHead(head_3031);
84        System.out.println("list setelah head dihapus: ");
85        printList(head_3031);
86        //hapus node terakhir
87        head_3031 = removeLastNode(head_3031);
88        printList(head_3031);
89        // Deleting node at position 2
90        int position_3031 = 2;
91        head_3031 = deleteNode(head_3031, position_3031);
92        //print list after deletion
93        System.out.println("list setelah posisi 2 dihapus: ");
94        printList(head_3031);
95    }
96 }

```

Gambar 2.4

Class HapusSLL digunakan untuk mengimplementasikan operasi penghapusan node pada struktur data Single Linked List. Terdapat beberapa method utama, yaitu deleteHead() untuk menghapus node pertama dengan cara memindahkan pointer head ke node berikutnya, removeLastNode() untuk menghapus node terakhir dengan menelusuri hingga node kedua terakhir, serta deleteNode() untuk menghapus node pada posisi tertentu dengan menyesuaikan pointer antar node. Setiap method menangani kondisi khusus seperti list kosong atau posisi yang tidak valid.

Selain itu, terdapat method printList() yang digunakan untuk menampilkan isi Linked List secara berurutan. Pada method main, dibuat sebuah Linked List awal kemudian dilakukan beberapa operasi penghapusan seperti menghapus node pertama, node terakhir, dan node pada posisi tertentu. Program ini menunjukkan bagaimana proses penghapusan data pada Linked List dapat dilakukan dengan mengatur pointer tanpa perlu menggeser elemen seperti pada array.

BAB III

PENUTUP

3.1 Kesimpulan

Pada praktikum pekan ke-5 telah dipelajari implementasi struktur data Single Linked List menggunakan bahasa pemrograman Java. Melalui beberapa program yang dibuat, dapat dipahami bahwa Linked List merupakan struktur data dinamis yang terdiri dari node-node yang saling terhubung melalui pointer, sehingga memungkinkan pengelolaan data yang lebih fleksibel dibandingkan array.

Selain itu, telah diimplementasikan berbagai operasi dasar pada Single Linked List seperti penambahan, penghapusan, pencarian, dan penelusuran data. Dari praktikum ini dapat disimpulkan bahwa Linked List sangat efisien dalam manipulasi data karena tidak memerlukan pergeseran elemen, namun memiliki keterbatasan dalam akses langsung terhadap data. Dengan memahami konsep ini, diharapkan mahasiswa dapat lebih mudah mengaplikasikan struktur data Linked List dalam berbagai permasalahan pemrograman.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Linked List Data Structure," GeeksforGeeks, 26 January 2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/linked-list-data-structure/>. [Accessed 05 Mei 2026].
- [2] Programiz, "Linked list Data Structure," Programiz, [Online]. Available: <https://www.programiz.com/dsa/linked-list>. [Accessed 05 Mei 2026].

LAPORAN TUGAS

A. Kode Program

Berikut kode program yang telah dibuat pada tugas praktikum pekan-5:

1. Pasien_2511533031

```

1 package pekan5_2511533031;
2
3 public class Pasien_2511533031 {
4     private String namaPasien_3031;
5     private String penyakit_3031;
6     private int nomorAntrian_3031;
7     private Pasien_2511533031 next_3031;
8     //Constructor untuk menginisialisasi semua atribut node Pasien.
9     public Pasien_2511533031(String namaPasien_3031, String penyakit_3031, int nomorAntrian_3031) {
10         this.namaPasien_3031 = namaPasien_3031;
11         this.penakit_3031 = penyakit_3031;
12         this.nomorAntrian_3031 = nomorAntrian_3031;
13         this.next_3031 = null; // default: belum ada node berikutnya
14     }
15     // Mengembalikan nama pasien
16     public String getNamaPasien_3031() {
17         return namaPasien_3031;
18     }
19     // Mengembalikan keluhan / penyakit pasien
20     public String getPenyakit_3031() {
21         return penyakit_3031;
22     }
23     // Mengembalikan nomor antrian pasien
24     public int getNomorAntrian_3031() {
25         return nomorAntrian_3031;
26     }
27     // Mengembalikan pointer ke node pasien berikutnya
28     public Pasien_2511533031 getNext_3031() {
29         return next_3031;
30     }
31     // Mengubah nama pasien
32     public void setNamaPasien_3031(String namaPasien_3031) {
33         this.namaPasien_3031 = namaPasien_3031;
34     }
35     // Mengubah keluhan / penyakit pasien
36     public void setPenyakit_3031(String penyakit_3031) {
37         this.penakit_3031 = penyakit_3031;
38     }
39     // Mengubah nomor antrian pasien
40     public void setNomorAntrian_3031(int nomorAntrian_3031) {
41         this.nomorAntrian_3031 = nomorAntrian_3031;
42     }
43     // Mengubah pointer ke node pasien berikutnya
44     public void setNext_3031(Pasien_2511533031 next_3031) {
45         this.next_3031 = next_3031;
46     }
47 }

```

Gambar 3.1

Class **Pasien** digunakan untuk merepresentasikan satu node dalam struktur data Single Linked List pada sistem antrian rumah sakit. Di dalam class ini terdapat beberapa atribut, yaitu namaPasien untuk menyimpan nama pasien, penyakit untuk menyimpan keluhan atau kondisi pasien, nomorAntrian sebagai identitas urutan pasien, serta next yang berfungsi sebagai pointer untuk menghubungkan ke node pasien berikutnya. Dengan adanya pointer ini, setiap data pasien dapat saling terhubung membentuk sebuah antrian yang dinamis.

Selain itu, class ini memiliki konstruktor yang digunakan untuk menginisialisasi seluruh atribut saat objek dibuat, serta dilengkapi dengan method getter dan setter. Method getter berfungsi untuk mengambil nilai dari setiap atribut, sedangkan setter digunakan untuk mengubah nilai

atribut tersebut. Class ini menjadi dasar dalam pembentukan struktur Linked List karena setiap pasien direpresentasikan sebagai node yang saling terhubung dalam sistem antrian.

2. RumahSakit_2511533031

```

1 package pekan5_2511533031;
2 import java.util.*;
3 public class RumahSakit_2511533031 {
4
5     // pointer ke node pertama (head)
6     private Pasien_2511533031 head_3031 = null;
7     // menyimpan nomor antrian (auto increment)
8     private int nomor_3031 = 0;
9     // Method untuk menambahkan pasien ke antrian (Insert di tail)
10    public void tambahPasien_3031(String nama_3031, String penyakit_3031) {
11        nomor_3031++; // nomor antrian bertambah otomatis
12        // membuat node pasien baru
13        Pasien_2511533031 baru_3031 = new Pasien_2511533031(nama_3031, penyakit_3031, nomor_3031);
14        // jika antrian kosong, node baru menjadi head
15        if (head_3031 == null) {
16            head_3031 = baru_3031;
17        }
18        // jika tidak kosong, telusuri sampai node terakhir
19        else {
20            Pasien_2511533031 temp_3031 = head_3031;
21            while (temp_3031.getNext_3031() != null) {
22                temp_3031 = temp_3031.getNext_3031();
23            }
24            // hubungkan node terakhir dengan node baru
25            temp_3031.setNext_3031(baru_3031);
26        }
27        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + nomor_3031);
28    }
29    // Method untuk memanggil pasien (Delete Head)
30    public void panggilPasien_3031() {
31        // jika antrian kosong
32        if (head_3031 == null) {
33            System.out.println("Antrian kosong!");
34            return;
35        }
36        // tampilkan data pasien yang dipanggil
37        System.out.println("Memanggil: " + head_3031.getNamaPasien_3031() +
38            " | Keluhan: " + head_3031.getPenyakit_3031());
39        // pindahkan head ke node berikutnya
40        head_3031 = head_3031.getNext_3031();
41    }
42    // Method untuk menampilkan seluruh antrian (Display)
43    public void tampilkanAntrian_3031() {
44        // jika antrian kosong
45        if (head_3031 == null) {
46            System.out.println("Antrian kosong!");
47            return;
48        }
49        Pasien_2511533031 temp_3031 = head_3031;
50        System.out.println("=== Daftar Antrian ===");
51        // menelusuri dari head sampai null
52        while (temp_3031 != null) {
53            System.out.println(
54                temp_3031.getNomorAntrian_3031() + " - " +
55                temp_3031.getNamaPasien_3031() + " (" +
56                temp_3031.getPenyakit_3031() + ")");
57            temp_3031 = temp_3031.getNext_3031();
58        }
59    }
60    // Method untuk mencari pasien berdasarkan nama (Search)
61    public void cariPasien_3031(String nama_3031) {
62        Pasien_2511533031 temp_3031 = head_3031;
63        boolean ketemu_3031 = false;
64        // telusuri seluruh linked list
65        while (temp_3031 != null) {
66            // pencarian tidak membedakan huruf besar/kecil
67            if (temp_3031.getNamaPasien_3031().equalsIgnoreCase(nama_3031)) {
68                System.out.println("Pasien ditemukan: " +
69                    temp_3031.getNamaPasien_3031() +
70                    " | Keluhan: " + temp_3031.getPenyakit_3031() +
71                    " | No: " + temp_3031.getNomorAntrian_3031());
72                ketemu_3031 = true;
73                break;
74            }
75            temp_3031 = temp_3031.getNext_3031();
76        }
77        // jika tidak ditemukan
78        if (!ketemu_3031) {
79            System.out.println("Pasien tidak ditemukan.");
80        }
81    }
82    // Method untuk menampilkan status antrian
83    public void status_3031() {
84

```

```

85 // jika antrian kosong
86 if (head_3031 == null) {
87     System.out.println("Antrian kosong!");
88     return;
89 }
90 int jumlah_3031 = 0;
91 Pasien_2511533031 temp_3031 = head_3031;
92 // menghitung jumlah pasien
93 while (temp_3031 != null) {
94     jumlah_3031++;
95     temp_3031 = temp_3031.getNext_3031();
96 }
97 // menampilkan hasil
98 System.out.println("Jumlah pasien: " + jumlah_3031);
99 System.out.println("Pasien terdepan: " + head_3031.getNamaPasien_3031());
100 }
101 // Method utama (main program)
102 public static void main(String[] args) {
103     Scanner sc_3031 = new Scanner(System.in);
104     RumahSakit_2511533031 rs_3031 = new RumahSakit_2511533031();
105     int pilih_3031;
106     do {
107         System.out.println("=== Antrian Rumah Sakit NIK: 2511533031 ===");
108         System.out.println("1. Daftarkan Pasien (Insert)");
109         System.out.println("2. Panggil Pasien (Delete Head)");
110         System.out.println("3. Tampilkan Antrian (Display)");
111         System.out.println("4. Cari Pasien (Search)");
112         System.out.println("5. Cek Status Antrian");
113         System.out.println("6. Keluar");
114         System.out.print("Pilihan: ");
115         pilih_3031 = sc_3031.nextInt();
116         sc_3031.nextLine();
117         switch (pilih_3031) {
118             case 1:
119                 System.out.print("Masukkan Nama Pasien : ");
120                 String nama_3031 = sc_3031.nextLine();
121                 System.out.print("Masukkan Keluhan : ");
122                 String penyakit_3031 = sc_3031.nextLine();
123                 rs_3031.tambahPasien_3031(nama_3031, penyakit_3031);
124                 break;
125             case 2:
126                 rs_3031.panggilPasien_3031();
127                 break;
128             case 3:
129                 rs_3031.tampilkanAntrian_3031();
130                 break;
131             case 4:
132                 System.out.print("Masukkan nama yang dicari : ");
133                 String cari_3031 = sc_3031.nextLine();
134                 rs_3031.cariPasien_3031(cari_3031);
135                 break;
136             case 5:
137                 rs_3031.status_3031();
138                 break;
139             case 6:
140                 System.out.println("Program selesai");
141                 break;
142             default:
143                 System.out.println("Pilihan tidak valid!");
144         }
145     } while (pilih_3031 != 6);
146     sc_3031.close();
147 }
148 }

```

Gambar 3.2

Class RumahSakit digunakan untuk mengelola sistem antrian pasien dengan menggunakan struktur data Single Linked List. Pada class ini terdapat atribut head yang berfungsi sebagai penunjuk ke node pertama dalam antrian, serta variabel nomor yang digunakan untuk memberikan nomor antrian secara otomatis. Setiap pasien yang datang akan ditambahkan ke bagian akhir antrian melalui method tambahPasien(), sedangkan pasien yang dilayani akan dihapus dari bagian depan menggunakan method panggilPasien(), sehingga sistem berjalan sesuai konsep FIFO (First In First Out).

Selain itu, class ini juga menyediakan beberapa method lain seperti tampilkanAntrian() untuk menampilkan seluruh data pasien, cariPasien() untuk mencari pasien berdasarkan nama tanpa membedakan huruf besar

dan kecil, serta status() untuk menampilkan jumlah pasien dan pasien yang berada di posisi terdepan. Pada method main, disediakan menu interaktif yang memungkinkan pengguna untuk melakukan berbagai operasi terhadap antrian secara langsung, sehingga program dapat mensimulasikan proses antrian rumah sakit secara sederhana dan dinamis.

B. Output

```
<terminated> RumahSakit_2511533031 [Java Application] C:\Users\User\p2\pool\plugins\org.ecli
=== Antrian Rumah Sakit NIM: 2511533031 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : nabil
Masukkan Keluhan      : demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533031 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : pian
Masukkan Keluhan      : pusing
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511533031 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
=== Daftar Antrian ===
1. nabil (demam)
2. pian (pusing)

=== Antrian Rumah Sakit NIM: 2511533031 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
```

Gambar 3.3

```
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama yang dicari : nabil
Pasien ditemukan: nabil | Keluhan: demam | No: 1

=== Antrian Rumah Sakit NIM: 2511533031 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah pasien: 2
Pasien terdepan: nabil

=== Antrian Rumah Sakit NIM: 2511533031 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil: nabil | Keluhan: demam

=== Antrian Rumah Sakit NIM: 2511533031 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Program selesai
```

Gambar 3.4